

Yes. Based on your **EP-01 Smart Meeting Action Extractor**, the best approach is **not** to build a simple “paste notes → LLM → table” application. The base problem statement is exactly that conversion, but the winning version should become a **Meeting Intelligence + Action Execution Platform**.

Your uploaded specification defines the core objective as converting messy meeting notes into **decisions, action items, owners, due dates and risks**. The Imagine SDK documentation is especially useful for this project because it explicitly supports **chat, image processing, audio transcription, embeddings, reranking, tool calling, RAG, agents, guarded LLMs and logging**.

Below is the architecture and roadmap I would recommend.

1. The final vision

Instead of:

Meeting Notes → AI → Action Items

build:

**Any Meeting Input → Multimodal Understanding → Meeting Intelligence →
Action/Decision/Risk Extraction → Contextual Memory → Dependency Analysis →
Human Verification → External Execution**

The system should accept:

1. 🖋️ Typed/pasted notes
2. 📝 Handwritten meeting notes
3. 🎤 Voice/audio recording
4. 📄 PDF meeting minutes
5. 🖼️ Images/screenshots/whiteboards
6. 📋 Previous meeting summaries

and produce:

- Meeting summary
- Decisions
- Action items
- Owners
- Due dates
- Priority
- Risks
- Dependencies

- Confidence
- Evidence/source text
- Project/team classification
- Historical context
- Contradictory decisions
- Overdue/unresolved items

Then allow the user to **actually execute the outcomes**:

- Create GitHub issue
 - Create Trello card
 - Create Notion task
 - Generate calendar event
 - Export CSV/JSON/ICS
 - Generate follow-up message
-

2. Most important architectural decision

I recommend:

Frontend

React + Vite + Tailwind CSS

Backend

Python FastAPI

AI

Qualcomm Imagine SDK

Database

SQLite initially

Vector database

ChromaDB

OCR

Tesseract / EasyOCR

Audio

Imagine SDK transcription if available to your account, otherwise local Whisper/faster-whisper as a free fallback.

Visualization

Mermaid.js + Chart.js

Integrations

- GitHub REST API
- Google Calendar / [.ics](#)
- Notion API
- Trello API

Deployment

Initially local → later deploy backend/frontend.

This is much better than putting everything inside Streamlit. Your existing material already identifies FastAPI + React/Next.js as the more modular production architecture.

3. Complete architecture diagram

This is the architecture I would actually build.

flowchart TB

U[" USER"]

subgraph INPUT["MULTIMODAL INPUT LAYER"]

TXT[" Typed / Pasted Notes"]

IMG[" Handwritten Notes / Images"]

AUD[" Audio / Voice Recording"]

PDF[" PDF / Documents"]

end

subgraph INGEST["INGESTION & PREPROCESSING"]

```
CLEAN["Text Cleaning"]
OCR["OCR Engine"]
STT["Speech-to-Text"]
PDFX["PDF Text Extraction"]
NORMAL["Unified Text Representation"]
end
```

```
subgraph AI["QUALCOMM AI INTELLIGENCE LAYER"]
  LLM["Qualcomm Imagine SDK<br/>LLM"]
  EXTRACT["Structured Information Extraction"]
  CLASSIFY["Project / Team Classification"]
  SENT["Sentiment / Meeting Tone"]
  CONF["Confidence Estimation"]
end
```

```
subgraph INTELLIGENCE["MEETING INTELLIGENCE ENGINE"]
  SUMMARY["Meeting Summary"]
  DEC["Decision Extraction"]
  ACTION["Action Item Extraction"]
  RISK["Risk Detection"]
  OWNER["Owner Resolution"]
  DATE["Due Date Resolution"]
  DEP["Dependency Detection"]
  DUP["Duplicate / Conflict Detection"]
  QUALITY["Meeting Quality Score"]
end
```

```
subgraph MEMORY["CONTEXTUAL MEMORY / RAG"]
  EMB["Imagine Embeddings"]
  VECTOR["ChromaDB"]
  HISTORY["Past Meetings"]
  CONTEXT["Relevant Historical Context"]
end
```

```
subgraph VERIFICATION["HUMAN VERIFICATION"]
  REVIEW["Review & Edit"]
  EVIDENCE["Evidence / Source Highlight"]
  APPROVE["Approve / Reject"]
end
```

```
subgraph EXECUTION["ACTION EXECUTION"]
  GH["GitHub Issues"]
  GC["Google Calendar / ICS"]
  NOTION["Notion"]
end
```

```
TRELLO["Trello"]  
EXPORT["CSV / JSON / Markdown"]  
end
```

```
subgraph UI["WEB DASHBOARD"]  
  DASH["Meeting Dashboard"]  
  TASKS["Action Tracker"]  
  GRAPH["Dependency Graph"]  
  HISTORYUI["Meeting History"]  
  ANALYTICS["Analytics"]  
end
```

```
U --> TXT  
U --> IMG  
U --> AUD  
U --> PDF
```

```
TXT --> CLEAN  
IMG --> OCR  
AUD --> STT  
PDF --> PDFX
```

```
OCR --> NORMAL  
STT --> NORMAL  
PDFX --> NORMAL  
CLEAN --> NORMAL
```

```
NORMAL --> LLM
```

```
LLM --> EXTRACT  
LLM --> CLASSIFY  
LLM --> SENT  
LLM --> CONF
```

```
EXTRACT --> SUMMARY  
EXTRACT --> DEC  
EXTRACT --> ACTION  
EXTRACT --> RISK
```

```
ACTION --> OWNER  
ACTION --> DATE  
ACTION --> DEP
```

```
DEC --> DUP
```

ACTION --> DUP
RISK --> QUALITY
SENT --> QUALITY
CONF --> QUALITY

NORMAL --> EMB
EMB --> VECTOR
VECTOR --> HISTORY
HISTORY --> CONTEXT
CONTEXT --> LLM

SUMMARY --> REVIEW
DEC --> REVIEW
ACTION --> REVIEW
RISK --> REVIEW
DEP --> REVIEW
CONF --> EVIDENCE

REVIEW --> APPROVE

APPROVE --> GH
APPROVE --> GC
APPROVE --> NOTION
APPROVE --> TRELLO
APPROVE --> EXPORT

SUMMARY --> DASH
ACTION --> TASKS
DEP --> GRAPH
HISTORY --> HISTORYUI
QUALITY --> ANALYTICS

This should be your **main architecture diagram** for understanding the project.

4. What each layer actually does

Layer 1 — Input

The user should see something like:

Smart Meeting Extractor

How would you like to provide your meeting?

[Paste Text]

[Upload PDF]

[Upload Handwritten Image]

[Upload Audio]

[Upload Document]

Project: [Qualcomm AI Project ▼]

Team: [Development Team ▼]

[Analyze Meeting]

The important concept is:

All inputs eventually become normalized text + metadata.

That means the AI extraction engine doesn't care whether the input came from a PDF, image or voice recording.

5. Input 1 — Normal text

This is your MVP.

Example:

Meeting: Project Alpha

Rahul will finish the authentication module by Friday.

Priya said the database migration may cause downtime.

We decided to use PostgreSQL.

Amit will prepare the deployment documentation next Monday.

The system converts this into:

```
{  
  "summary": "...",  
  "decisions": [],  
  "action_items": [],  
  "risks": [],  
  "dependencies": []  
}
```

6. Input 2 — Handwritten OCR

This can be a very good differentiator.

Example:

User photographs:

Project Alpha Meeting

Rahul - auth by Fri

Priya - DB migration risk

Amit - deployment docs Monday

Decision:

Use PostgreSQL

Pipeline:

Image

↓

Image preprocessing

↓

OCR

↓

Clean OCR text

↓

Qualcomm LLM

↓

Structured meeting intelligence

Important caveat

Do **not** claim:

"Our OCR perfectly understands handwriting."

Handwriting OCR is inherently noisy.

Instead:

Add an OCR verification screen

OCR Extracted Text

Rahul - auth by Fri

Priya - DB migration risk

Amit - deployment docs Monday

↓

[Edit OCR Text]

↓

[Analyze Meeting]

This makes the system much more reliable.

7. Input 3 — Voice-to-text

Architecture:

Microphone / Audio File

↓

Audio Upload

↓

Speech-to-Text Engine

↓

Transcript

↓

Speaker Detection

↓

Qualcomm LLM

You can use the Imagine ecosystem for transcription because the supplied SDK tutorials explicitly document “**Transcribe audio**” as a supported capability.

However, build a fallback:

Primary:

Imagine SDK transcription

Fallback:

Local Whisper / faster-whisper

That prevents your entire demo from failing if the cloud transcription endpoint isn't available/configured.

8. Input 4 — PDF

Pipeline:

PDF

↓

PDF parser

↓

Extract text

↓

Clean text

↓

LLM

For scanned PDFs:

PDF

↓

Detect scanned pages

↓

OCR

↓

Text

↓

LLM

This gives you:

Digital PDF

PDF → text extraction → AI

and

Scanned PDF

PDF → OCR → AI

9. The core AI extraction engine

This is the heart of the project.

Your LLM shouldn't simply receive:

"Summarize this meeting."

Instead, give it a strict schema.

For example:

```
{
  "meeting_summary": "...",

  "decisions": [
    {
      "decision": "...",
      "evidence": "...",
      "confidence": 0.94
    }
  ],

  "action_items": [
    {
      "id": "A001",
      "task": "...",
      "owner": "...",
      "due_date": "...",
      "priority": "HIGH",
      "status": "OPEN",
      "confidence": 0.91,
      "evidence": "...",
      "dependencies": []
    }
  ]
}
```

```
],  
  
"risks": [  
  {  
    "risk": "...",  
    "severity": "HIGH",  
    "mitigation": "...",  
    "confidence": 0.84  
  }  
]  
}
```

10. One feature I strongly recommend: Evidence-based AI

This could be one of your strongest differentiators.

Instead of displaying:

Action: Rahul will complete authentication by Friday.

Display:

Action: Rahul will complete authentication by Friday

Confidence: 94%

Evidence: “Rahul will finish the authentication module by Friday.”

And allow:

[View source]

The original sentence gets highlighted.

This addresses a major problem with LLM applications:

Where did the AI get this conclusion from?

Your system becomes explainable.

11. Add "AI confidence"

Every extracted item should have:

Action

Rahul → Authentication

Confidence



Evidence

"Rahul will complete auth by Friday."

Use:

- 90–100 → High confidence
- 70–89 → Medium
- <70 → Needs review

But be careful:

Do not present an LLM's self-reported confidence as statistically calibrated probability.

Call it:

Extraction Confidence

rather than claiming:

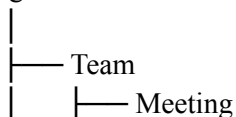
94% probability that this is correct.

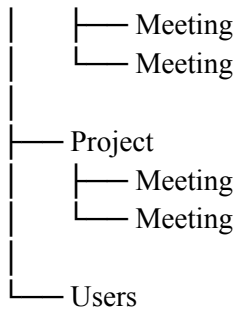
12. Project and team grouping

This should become a major application feature.

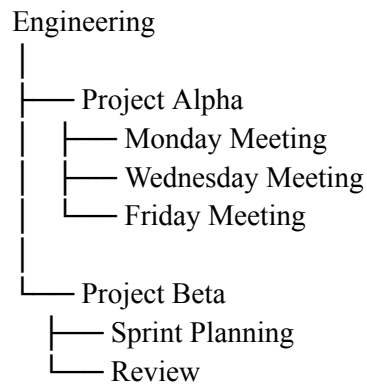
Database structure:

Organization





Example:



Now your application becomes a **persistent workspace**, rather than a one-time AI demo.

13. RAG — contextual history

This is where the project becomes significantly more advanced.

Imagine:

Previous meeting

"We decided to use PostgreSQL."

Next meeting:

"Let's move to MongoDB."

Your system should say:

 **Potential Decision Conflict**

Current decision:
Move to MongoDB

Previous decision:
Use PostgreSQL

Previous meeting:
September 29

Would you like to review the previous decision?

That's much more impressive than generic summarization.

The Imagine documentation specifically includes **embeddings and RAG with ChromaDB**, so this is also well aligned with the platform rather than an unrelated add-on.

14. RAG architecture

flowchart LR

```
M["Past Meeting"] --> E["Imagine Embedding"]
E --> V["ChromaDB"]

N["New Meeting"] --> NE["Generate Embedding"]
NE --> S["Similarity Search"]

S --> R["Relevant Past Meetings"]

R --> L["Qualcomm LLM"]
N --> L

L --> O["Context-Aware Analysis"]
```

Example:

```
New Meeting
↓
"What did we decide about database?"
↓
Embedding
↓
Vector search
```

↓
Top 5 relevant meetings
↓
Qualcomm LLM
↓
Answer

15. Dependency graph

This is another feature I would absolutely include.

Suppose:

Amit → API specification
Rahul → Backend implementation
Priya → Testing

AI detects:

API specification
↓
Backend implementation
↓
Testing

Then display:

flowchart LR

```
A["API Specification"] --> B["Backend Implementation"]  
B --> C["Testing"]  
C --> D["Deployment"]
```

Now the judge can immediately see:

This isn't merely summarization.

It's **project intelligence**.

16. Dependency risk detection

Take it one step further.

Suppose:

Task A:
API Specification
Due: Oct 10

Task B:
Backend Implementation
Due: Oct 8

Dependency:
B depends on A

Your system should detect:

 **Schedule Conflict**

because:

API Spec → Oct 10
Backend → Oct 8

The system can say:

Backend implementation may be blocked because its dependency is scheduled after the dependent task.

This is a **deterministic business rule**, not an LLM hallucination.

That's important.

17. Don't make everything AI

This is one of my biggest recommendations.

Use:

AI for

- Understanding language
- Extracting actions

- Identifying decisions
- Identifying risks
- Understanding dependencies
- Summarization
- Historical context

Deterministic code for

- Due-date calculations
- Overdue detection
- Priority sorting
- Dependency validation
- Calendar generation
- Duplicate detection
- Database operations
- Integration APIs
- Permission checks

This makes the application much more reliable.

18. Meeting sentiment / tone

You mentioned:

voice sentiment

I would slightly change the implementation.

Don't claim:

"We detect the actual emotional state of the people."

Instead:

Meeting Tone & Conflict Analysis

Analyze the transcript for:

- Positive
- Neutral
- Concern
- Conflict

- Escalation
- Risk-heavy discussion

Example:

Meeting Tone



This is much easier to defend.

You can also detect phrases such as:

"I disagree..."

"I'm concerned..."

"This won't work..."

"We need to revisit..."

"That's blocked..."

and identify **conflict/risk signals**.

19. Meeting effectiveness score

This is a feature I would definitely add.

Calculate:

Meeting Effectiveness Score

based on deterministic metrics:

- + Decisions made
- + Action items assigned
- + Due dates specified
- + Risks identified
- + Low number of unresolved items
- + Clear ownership
- Unassigned tasks
- Missing deadlines
- Conflicting decisions

Example:

MEETING EFFECTIVENESS

78 / 100

Decision Clarity	88
Ownership	92
Deadline Coverage	71
Risk Coverage	83
Dependency Clarity	65

This gives judges an immediately understandable dashboard metric.

20. Workload imbalance detection

This is another unique feature.

Suppose:

Rahul → 9 tasks

Priya → 2 tasks

Amit → 1 task

System:

 **Potential workload imbalance**

Rahul		9
Priya		2
Amit		1

Don't claim it's objectively unfair.

Say:

"Potential workload concentration detected."

That's defensible.

21. Smart follow-up detection

This is excellent for the demo.

Previous meeting:

Rahul:

Complete API documentation by Friday.

Next meeting:

No mention of API documentation.

System:

 **Unresolved commitment**

Task:

API Documentation

Assigned:

Rahul

Original Due:

Oct 2

Status:

No completion evidence found

Last Mentioned:

Sept 28

This is where RAG becomes useful.

22. Action-item lifecycle

Don't stop at extraction.

Give each task:

Detected

↓

Reviewed
↓
Approved
↓
Assigned
↓
In Progress
↓
Completed

Database:

status:
DETECTED
REVIEWED
APPROVED
IN_PROGRESS
BLOCKED
COMPLETED
CANCELLED

Now your application becomes a lightweight **meeting-to-execution system**.

23. Integrations

You specifically requested:

GitHub

Create issue:

Title:
Implement authentication module

Assignee:
Rahul

Due:
Oct 10

Priority:
High

Source:

Project Alpha Meeting – Oct 4

GitHub's authenticated REST API provides substantially higher limits than unauthenticated requests; GitHub documents 5,000 requests/hour for authenticated users. ([GitHub Docs](#))

For a hackathon prototype, that is more than enough.

24. Google Calendar

Don't make Google Calendar mandatory.

Use two options:

Option A — completely local

Generate:

meeting_task.ics

The user downloads/imports it into their calendar.

No Google API required.

Option B

Google Calendar API.

Google currently documents standard Calendar API usage as having no additional cost within its limits, although Google also documents quota/billing changes later in 2026. ([Google for Developers](#))

For your hackathon:

ICS export should be your guaranteed-free fallback.

25. Notion

Create:

Notion Database

Task | Owner | Due Date | Priority | Project | Status

The Notion API supports custom internal connections/tokens for workspace access. ([Notion](#))

Again:

Primary:

Notion API

Fallback:

Markdown/CSV export

26. Trello

Convert:

Action Item

into:

Trello Card

Example:

TO DO

- └─ API Documentation
- └─ Database Migration
- └─ Testing

IN PROGRESS

- └─ Authentication

DONE

- └─ UI Design

Trello provides REST API access with documented rate limits, so your small hackathon workload should be nowhere near those limits. ([Atlassian Developer](#))

27. Your "free-first" technology strategy

This is important.

Don't design the project around ten paid SaaS APIs.

Use:

Feature	Recommended technology	Cost strategy
LLM	Qualcomm Imagine SDK	Platform-provided
Text extraction	Python	Free
OCR	Tesseract/EasyOCR	Free/open-source
PDF	PyMuPDF/pypdf	Free
Audio	Imagine transcription / local Whisper	Free fallback
Database	SQLite	Free
Vector DB	ChromaDB	Free
Embeddings	Imagine SDK	Platform
Graph	Mermaid.js	Free
Charts	Chart.js	Free
GitHub	GitHub REST	Free within limits
Calendar	.ics	Completely free
Notion	Notion API	Free API access
Trello	Trello API	Free API access
Authentication	Local/session auth initially	Free
Hosting	Local demo	Free

28. Features I would NOT prioritize

Don't try to implement everything.

Avoid spending time on:

- ✗ Full enterprise authentication
- ✗ Complex RBAC
- ✗ Mobile application
- ✗ Video conferencing integration
- ✗ Real-time collaborative editing
- ✗ Kubernetes
- ✗ Microservices
- ✗ Payment system
- ✗ Complex DevOps
- ✗ Training your own LLM

These consume time without substantially improving your hackathon score.

29. Your strongest unique features

If I had to choose only **10**, I would use:

Tier 1 — MUST HAVE

1. **Multimodal input**
 - Text
 - PDF
 - Image
 - Audio
 2. **Structured meeting extraction**
 3. **Evidence-backed results**
 4. **Human verification**
 5. **Project/team organization**
-

Tier 2 — Winning differentiators

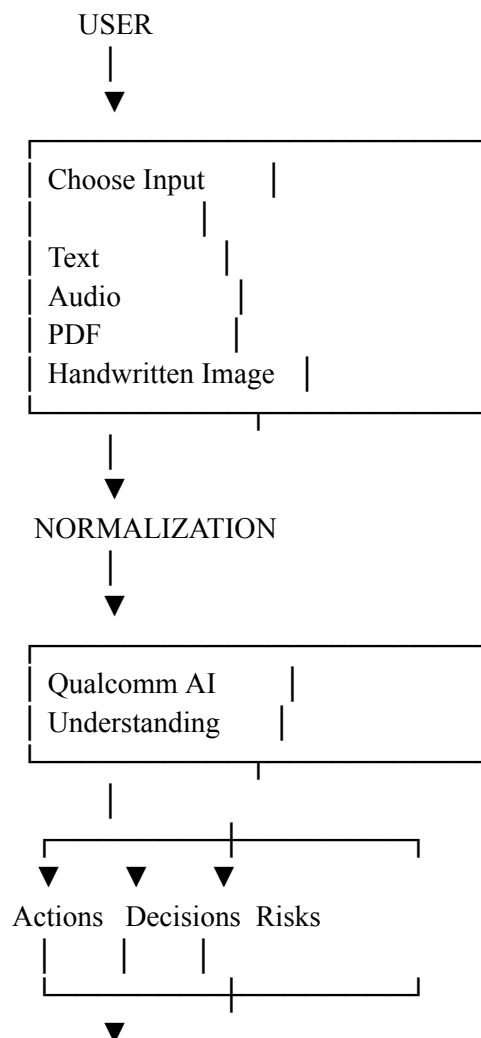
6. **RAG historical memory**
7. **Decision conflict detection**
8. **Dependency graph + schedule conflict detection**
9. **Unresolved commitment detection**
10. **Action execution integrations**

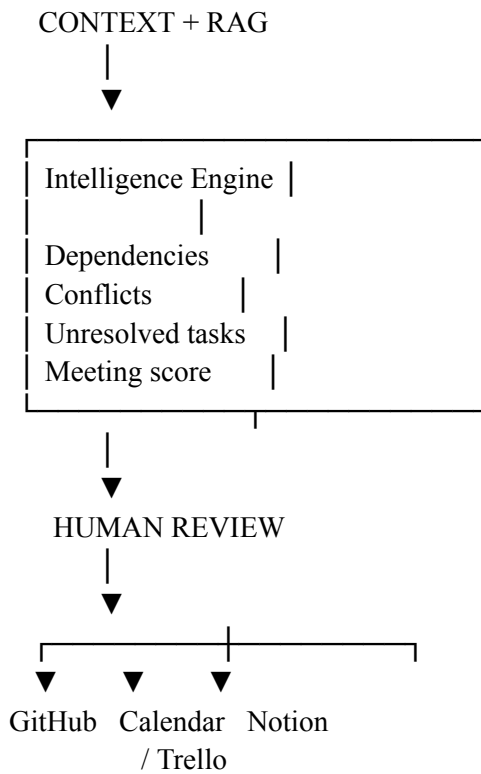
Tier 3 — Extra polish

- 11. Meeting effectiveness score
 - 12. Workload concentration
 - 13. Meeting tone/conflict analysis
 - 14. Smart follow-up generation
 - 15. Export to multiple formats
-

30. The complete user journey

Your final demo should look like this:





31. Recommended database design

Start with SQLite.

users

id
name
email

teams

id
name
description

projects

id
name
description
team_id

meetings

id
title
project_id
team_id
meeting_date
input_type
raw_text
summary
tone
effectiveness_score
created_at

decisions

id
meeting_id
decision
evidence
confidence

action_items

id
meeting_id
project_id
task
owner
due_date
priority
status
confidence
evidence

risks

id
meeting_id
risk
severity
mitigation
confidence

dependencies

id

source_task_id
target_task_id
dependency_type

integrations

id
user_id
provider
access_token

meeting_embeddings

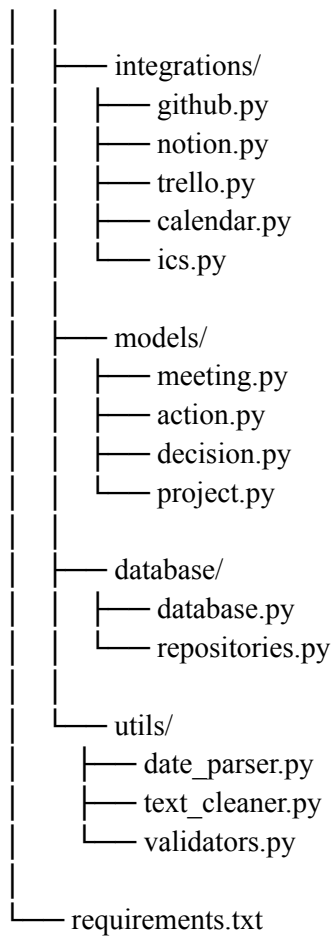
meeting_id
embedding_id

32. Recommended backend architecture

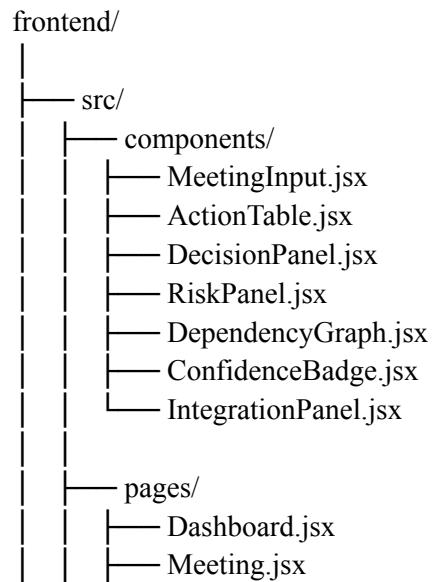
Don't put everything in `main.py`.

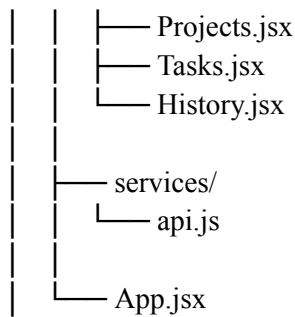
Use:

```
backend/  
├── app/  
│   ├── main.py  
│   ├── api/  
│   │   ├── meeting_routes.py  
│   │   ├── project_routes.py  
│   │   ├── task_routes.py  
│   │   ├── rag_routes.py  
│   │   └── integration_routes.py  
│   └── services/  
│       ├── ai_service.py  
│       ├── extraction_service.py  
│       ├── ocr_service.py  
│       ├── transcription_service.py  
│       ├── rag_service.py  
│       ├── dependency_service.py  
│       ├── sentiment_service.py  
│       └── scoring_service.py
```



33. Frontend architecture





34. Development roadmap

Do **not** build all features simultaneously.

Phase 0 — Environment

Set up:

Python
Node.js
VS Code
Git
Virtual Environment
Imagine SDK

The supplied Imagine documentation says the SDK is installed as a Python package/wheel and exposes synchronous [ImagineClient](#) and asynchronous [ImagineAsyncClient](#).

Configure:

IMAGINE_API_KEY
IMAGINE_ENDPOINT_URL

The documentation explicitly instructs users to configure these values and warns not to expose API keys or commit them to Git.

35. Phase 1 — Basic AI engine

Build:

Text
↓
FastAPI
↓
Imagine SDK
↓
Structured JSON
↓
Frontend

Do not move forward until this works reliably.

36. Phase 2 — Database

Add:

SQLite
↓
Meetings
Projects
Teams
Actions
Decisions
Risks

Now your application has persistence.

37. Phase 3 — Multimodal input

Add one by one:

3.1 PDF

PDF → Text → AI

3.2 Image

Image → OCR → Text → AI

3.3 Audio

Audio → STT → Text → AI

Don't build all three simultaneously.

38. Phase 4 — Evidence + confidence

Add:

Action

Confidence

Evidence

This is a major reliability improvement.

39. Phase 5 — Dependency engine

Add:

Task A

↓

Task B

↓

Task C

Then:

Dependency graph

+

Deadline validation

40. Phase 6 — RAG

Implement:

Meeting

↓
Embedding
↓
ChromaDB

Then:

New Meeting
↓
Similarity Search
↓
Relevant Past Meetings
↓
LLM
↓
Historical Context

The Imagine tutorials explicitly include an example of RAG with ChromaDB, including ingestion and query phases.

41. Phase 7 — Intelligence

Add:

Decision conflicts

Previous:
Use PostgreSQL

Current:
Use MongoDB

→ Conflict detected

Unresolved tasks

Task from previous meeting
+
No completion evidence
=
Follow-up required

Workload concentration

Rahul → 8

Priya → 2

Amit → 1

Meeting effectiveness

78/100

42. Phase 8 — Integrations

Do them in this order:

1. GitHub

Most relevant for technical judges.

2. .ics

Almost zero complexity and no external API dependency.

3. Trello

Very visual.

4. Notion

Good productivity demonstration.

5. Google Calendar API

Only after the first four work.

43. Phase 9 — Human-in-the-loop

This is extremely important.

Never automatically create 15 GitHub issues from an LLM response.

Instead:

AI Extracted 7 Actions

- ☒ API documentation
- ☒ Database migration
- ☐ Testing
- ☒ Deployment

[Approve Selected]

[Reject]

[Edit]

Then:

Approved

↓

Integration

This makes the system safer and more realistic.

44. Phase 10 — Demo mode

Create a **Demo Workspace**.

Preload:

Project Alpha
Engineering Team
5 previous meetings
20 historical action items
10 decisions

Then demonstrate:

Meeting 1

We will use PostgreSQL.

Meeting 2

PostgreSQL migration may be delayed.

Meeting 3

Let's switch to MongoDB.

Your application detects:

⚠ Previous decision conflict

Then show:

Relevant historical meeting

Then create:

GitHub issue

Then show:

Dependency graph

This would make an excellent hackathon demo.

45. One feature that could really differentiate you

I would call it:

Meeting Continuity Engine

Instead of treating every meeting independently:

Meeting 1



Meeting 2



Meeting 3



Meeting 4

the system understands the **evolution of the project**.

It can answer:

What changed since the last meeting?

Which decisions were reversed?

Which action items are still unresolved?

Which risks keep recurring?

Who currently owns the most work?

Which commitments were missed?

What should we discuss in the next meeting?

That turns your project from:

AI meeting summarizer

into:

AI project continuity assistant.

46. Add a "Prepare Next Meeting" button

This is an excellent final feature.

Button:

Prepare Next Meeting

Output:

NEXT MEETING AGENDA

1. Review overdue API documentation
2. Resolve PostgreSQL/MongoDB decision
3. Discuss database migration risk
4. Review Rahul's pending tasks
5. Confirm deployment date

Unresolved items:
3

High-priority risks:
2

Previous commitments:
4

This is very compelling because the system doesn't just record what happened.

It tells the team:

What should happen next?

47. Final feature map

I would present the product as five major capabilities:

SMART MEETING INTELLIGENCE	
1. CAPTURE	
Text • Audio • PDF • Image	
2. UNDERSTAND	
Summary • Decisions • Actions • Risks	
3. REMEMBER	
RAG • History • Context	
4. REASON	
Dependencies • Conflicts • Risks	
5. EXECUTE	
GitHub • Calendar • Notion • Trello	

48. The final architecture you should aim for

flowchart TB

A["USER"]

A --> B["MULTIMODAL CAPTURE"]

B --> B1["Text"]

B --> B2["PDF"]

B --> B3["Handwritten Image"]

B --> B4["Audio"]

B1 --> C["NORMALIZATION"]

B2 --> C

B3 --> O["OCR"]

B4 --> S["SPEECH TO TEXT"]

O --> C

S --> C

C --> D["QUALCOMM IMAGINE SDK"]

D --> D1["LLM"]

D --> D2["Embeddings"]

D --> D3["Transcription"]

D --> D4["Tool Calling"]

D1 --> E["STRUCTURED EXTRACTION"]

E --> E1["Summary"]

E --> E2["Decisions"]

E --> E3["Actions"]

E --> E4["Risks"]

E --> E5["Owners"]

E --> E6["Due Dates"]

E --> F["INTELLIGENCE ENGINE"]

F --> F1["Dependency Detection"]

F --> F2["Decision Conflict"]

F --> F3["Unresolved Commitments"]

F --> F4["Workload Analysis"]

F --> F5["Meeting Effectiveness"]

F --> F6["Meeting Tone"]

D2 --> G["CHROMADB"]

G --> H["RAG CONTEXT"]

H --> D1

F --> I["HUMAN REVIEW"]

I --> J["APPROVAL"]

J --> K1["GitHub"]

J --> K2["Calendar / ICS"]

J --> K3["Notion"]

J --> K4["Trello"]

I --> L["DASHBOARD"]

L --> L1["Action Tracker"]

L --> L2["Decision Timeline"]

L --> L3["Dependency Graph"]

L --> L4["Risk Dashboard"]

L --> L5["Meeting History"]

L --> L6["Analytics"]

49. The most important implementation rule

Don't tell Kiro:

"Build the entire application."

in one shot.

It will generate a huge codebase with imaginary APIs, incorrect SDK methods and broken integrations.

Instead, make Kiro build it **phase by phase**.

The supplied SDK documentation gives you real capabilities such as chat, transcription, embeddings, tool calling, RAG, agents and guardrails. Therefore, your Kiro instructions should explicitly tell it:

Never invent an Imagine SDK method. Inspect the supplied SDK documentation and installed package before implementing it.

That is particularly important because the earlier draft in your file assumes methods such as `transcribe()` and `embeddings()`; the tutorials do establish those capabilities, but your generated code still needs to match the actual installed SDK API.

50. Master prompt structure for Kiro

When you are ready to generate the actual application, I recommend giving Kiro a prompt structured like this:

You are the lead software architect and senior full-stack AI engineer.

Build EP-01: Smart Meeting Action Extractor.

IMPORTANT:

Do not generate the entire application in one step.

First inspect:

1. Existing project files
2. Imagine SDK wheel
3. Imagine SDK documentation
4. Installed Python package
5. Available SDK methods

Never invent SDK methods.

=====

PHASE 1 — FOUNDATION

=====

Build:

Frontend:

React + Vite + Tailwind

Backend:

FastAPI

Database:

SQLite + SQLAlchemy

AI:

Qualcomm Imagine SDK

Implement only:

Text input

↓

FastAPI

↓

Imagine SDK

↓

Structured JSON

↓

React dashboard

Do not implement integrations yet.

Verify the application works before proceeding.

PHASE 2 — DATABASE

Add:

Users

Teams

Projects

Meetings

Decisions

Actions

Risks

Dependencies

Create migrations/schema.

PHASE 3 — MULTIMODAL INPUT

Add:

1. Text

2. PDF

3. Image OCR

4. Audio transcription

All inputs must produce a common normalized MeetingDocument object.

PHASE 4 — AI EXTRACTION

Extract:

summary
decisions
action_items
owners
due_dates
priority
risks
mitigation
dependencies
confidence
evidence

The LLM must return structured JSON.

PHASE 5 — EXPLAINABILITY

Every AI-generated item must contain:

confidence
evidence
source reference

Allow users to edit/reject AI results.

PHASE 6 — PROJECT MEMORY

Implement:

Imagine embeddings
ChromaDB

RAG

Store previous meetings.

For every new meeting:

retrieve relevant previous meetings.

Detect:

previous decisions

conflicting decisions

unresolved commitments

recurring risks

PHASE 7 — INTELLIGENCE

Implement deterministic services for:

dependency analysis

deadline conflicts

overdue tasks

workload concentration

meeting effectiveness

Do NOT ask the LLM to calculate deterministic metrics.

PHASE 8 — VISUALIZATION

Implement:

Action dashboard

Risk dashboard

Decision timeline

Dependency graph

Meeting history

Workload chart

Meeting effectiveness score

PHASE 9 — INTEGRATIONS

Implement independently:

GitHub

ICS Calendar

Trello

Notion

Google Calendar

Each integration must have:

service class

configuration

authentication

error handling

test endpoint

enable/disable option

Never expose API tokens in frontend code.

PHASE 10 — MEETING CONTINUITY

Add:

"Prepare Next Meeting"

Generate:

unresolved tasks

overdue tasks

previous decisions

decision conflicts

recurring risks

recommended agenda

PHASE 11 — DEMO MODE

Create realistic demo data:

3 projects
3 teams
5 historical meetings
20 action items
10 decisions
5 risks

Create a complete demo workflow.

QUALITY REQUIREMENTS

Use:

clean architecture
type hints
Pydantic
error handling
logging
environment variables
API validation
modular services
unit tests

Never hardcode API keys.

Do not implement features using imaginary APIs.

After each phase:

1. Generate files
 2. Explain changes
 3. Run/test
 4. Fix errors
 5. Only then continue
-

51. What your final demo should demonstrate

If you have around **5–7 minutes**, don't show every feature.

Use this story:

0:00–0:30 — Problem

"Meeting outcomes are often trapped inside messy notes, PDFs, screenshots and recordings."

0:30–1:15 — Multimodal input

Upload:

handwritten meeting note

and show:

OCR → structured meeting

1:15–2:00 — AI extraction

Show:

Summary

Decisions

Actions

Risks

2:00–2:45 — Explainability

Click an action:

Confidence

Evidence

Source

2:45–3:30 — RAG

Show:

"This decision conflicts with a decision from last week's meeting."

3:30–4:15 — Dependency intelligence

Show:

API → Backend → Testing → Deployment

and:

"Deployment may be delayed because the API specification is overdue."

4:15–5:00 — Execution

Click:

Create GitHub Issues

or

Add to Trello

5:00–5:45 — Meeting continuity

Click:

Prepare Next Meeting

and show the recommended agenda.

That is a much stronger story than simply:

"Here is an AI-generated summary."

My recommended final scope

If time becomes limited, prioritize in this exact order:

P0 — Core

- Text
- Qualcomm LLM
- Structured extraction
- Dashboard
- SQLite

P1 — Differentiation

- PDF
- Handwritten OCR
- Audio
- Evidence + confidence

- Project/team grouping

P2 — Winning features

- RAG
- Decision conflict detection
- Dependency graph
- Unresolved commitment detection
- Meeting effectiveness

P3 — Execution

- GitHub
- ICS
- Trello
- Notion

P4 — Polish

- Google Calendar
- Meeting tone
- Workload analytics
- Advanced visualizations

The key is that **P0 must be rock-solid before P1**, and **P1 before P2**. Don't sacrifice a reliable core application just to claim nine integrations.

And importantly, the Qualcomm component should remain central rather than decorative: the Imagine SDK is specifically intended to simplify GenAI applications and workflow automation, and its documented capabilities map unusually well to your planned architecture.

If you build it this way, I would describe the final product not as an **“AI Meeting Summarizer”**, but as:

“An AI-powered Meeting-to-Execution platform that captures meeting information from any format, understands decisions and commitments, remembers project history, detects risks and dependencies, and converts meeting outcomes into executable tasks.”

That positioning is considerably stronger for the Infosys/Qualcomm hackathon than the basic problem statement alone.